

Event-Driven Processing Simulator: Project Report

1. Executive Summary

The **Event-Driven Processing Simulator** is a simple demonstration of a backend utility designed to model and monitor asynchronous task execution in a multi-threaded environment. The system allows users to submit events via a REST API, which are then queued, processed by a managed pool of worker threads, and persisted for historical analysis. Its primary purpose is to provide a sandbox for testing system throughput, resource contention, and event lifecycle management in a controlled, observable ecosystem.

2. Design Patterns

The simulator is built on top of proven design patterns that are used to solve concurrency and queuing problems in computer science. Here is a list of the design patterns I have used to solve some of the problems that are brought about by this project.

- **Producer-Consumer Pattern:** The system decouples event submission (Producer) from event execution (Consumer). The API layer acts as the producer, accepting incoming requests and handing them off to the internal processing engine.
- **ExecutorService & Thread Pooling:** To prevent resource exhaustion, the application utilizes a custom-configured **ExecutorService**. This manages a fixed-size pool of worker threads, ensuring that the system does not create an unbounded number of threads, which would lead to context-switching overhead and memory issues.
- **BlockingQueue:** At the heart of the thread-safe communication is a **BlockingQueue**. When all worker threads are busy, incoming events are

parked in this queue. It handles the synchronization between the producer and consumer threads, ensuring no events are lost and that worker threads wait efficiently for new tasks.

3. Technology Stack

The project utilizes a modern Java-based stack to ensure maintainability and rapid deployment.

Component	Technology
Language	Java 17
Framework	Spring Boot 3.x
Data Persistence	Spring Data JPA
Database	H2 (In-Memory / File-based)
Dependency and Build	Maven
Containerization	Docker

4. Project Structure

The directory structure follows the standard Maven and Spring Boot conventions for clarity and modularity.

src/main/java/com/simulator/

- ├── config/ # Thread pool and security configurations
- ├── controller/ # REST API Endpoints
- ├── dto/ # Data Transfer Objects for API requests
- ├── exception/ # Global exception handling
- ├── model/ # JPA Entities (Event, Metric)

|— repository/ # Spring Data JPA Repositories
|— service/ # Business logic and Event Processor

5. Advance Java Concepts

A. Java Springboot and REST API networking

This project demonstrates Spring Boot's REST framework for building scalable, distributed systems that communicate over HTTP. Spring Boot simplifies the development of network-enabled applications by providing annotations and abstractions that handle HTTP request routing, JSON serialization, and client-server communication.

Networking process:

- Clients (web browser, curl, other services) send HTTP requests to the server
- Spring Boot routes requests to appropriate handler methods based on the URL path and HTTP method
- The server sends back JSON responses with the requested data or confirmation of actions
- This enables real-time communication between the frontend dashboard and backend processing system

Code Example: Rest Endpoints in **EventControler.java**:

```
Java
@RestController
@RequestMapping("/api/events")
@CrossOrigin(origins = "*")
public class EventController {

    /**
     * Submit a new event for processing.
     * POST /api/events
     * Accepts JSON: { "eventType": "CREATE_ORDER", "size": 5 }
     */
}
```

```

    */
    @PostMapping
    public ResponseEntity<Event> submitEvent(@RequestBody Map<String, Object>
payload) {
        String eventTypeStr = (String) payload.get("eventType");
        Integer size = ((Number) payload.get("size")).intValue();
        Event event = eventService.submitEvent(EventType.valueOf(eventTypeStr),
size);
        return ResponseEntity.status(HttpStatus.CREATED).body(event);
    }

    /**
     * Query events by status.
     * GET /api/events?status=completed
     * Returns JSON array of matching events
     */
    @GetMapping
    public ResponseEntity<List<Event>> getEvents(@RequestParam(required = false)
String status) {
        List<Event> events = (status != null)
            ?
eventService.getEventsByStatus(EventStatus.valueOf(status.toUpperCase()))
            : eventService.getAllEvents();
        return ResponseEntity.ok(events);
    }

    /**
     * Get system metrics over the network.
     * GET /api/events/metrics
     * Returns JSON with queue size, active workers, completion stats
     */
    @GetMapping("/metrics")
    public ResponseEntity<Map<String, Object>> getMetrics() {
        return ResponseEntity.ok(metricsService.getMetrics());
    }
}

```

Demonstrated Learning Outcomes:

- HTTP Protocol: How clients and servers communicate using standard web protocols
- JSON Serialization: Converting Java objects to/from JSON for network transmission
- Request-Response Pattern: Clients make requests and wait for server responses
- Statelessness: Each HTTP request is independent; the server doesn't maintain connection state
- Distributed Systems: Multiple clients can interact with a single backend service

B. Database persistence and access patterns

This project demonstrates Spring Data JPA and Hibernate's automatic use of prepared statements for secure, efficient database communication. While developers write simple Java method calls, Spring automatically generates SQL with parameterized queries that prevent SQL injection and improve performance through query plan caching.

How Prepared Statements Work in This Project

The Repository Pattern: The `EventRepository` interface extends `JpaRepository`, which Spring implements with prepared statement logic automatically. Method names like `findByStatus()` and `countByStatus()` tell Spring Data JPA to generate parameterized SQL queries behind the scenes.

Security Through Parameterization: When the application queries events by status, Spring Data JPA generates prepared statements like:

```
SQL
SELECT * FROM events WHERE status = ?
```

The `?` is a placeholder where the actual status value is bound as a parameter, preventing SQL injection attacks. This is far safer than string concatenation.

Code Example: Database Operations

Entity Definition (Event.java):

```
Java
@Entity
@Table(name = "events")
@Data
public class Event {

    @Id
    @Column(name = "id", columnDefinition = "VARCHAR(36)")
    private String id;

    @Enumerated(EnumType.STRING)
    @Column(name = "event_type", nullable = false)
    private EventType eventType;

    @Column(name = "status", nullable = false)
    private EventStatus status;

    @Column(name = "processing_time_ms", nullable = false)
    private Long processingTimeMs;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    @Column(name = "started_at")
    private LocalDateTime startedAt;

    @Column(name = "completed_at")
    private LocalDateTime completedAt;

    @Column(name = "result", length = 1000)
    private String result;
}
```

Repository Interface with JPA:

Java

@Repository

```
public interface EventRepository extends JpaRepository<Event, String> {  
    // Spring Data JPA generates prepared statement:  
    // SELECT * FROM events WHERE status = ?  
    List<Event> findByStatus(EventStatus status);  
  
    // Generates: SELECT * FROM events WHERE status = ? ORDER BY created_at DESC  
    List<Event> findByStatusOrderByCreatedAtDesc(EventStatus status);  
  
    // Generates: SELECT COUNT(*) FROM events WHERE status = ?  
    long countByStatus(EventStatus status);  
}
```

Example of Service Layer Using the EventRepository Interface:

Java

@Service

```
public class EventService {  
  
    private final EventRepository eventRepository;  
  
    /**  
     * Query events by status using prepared statements.  
     * Spring Data JPA automatically creates and caches the prepared statement.  
     */  
    public List<Event> getEventsByStatus(EventStatus status) {  
        return eventRepository.findByStatusOrderByCreatedAtDesc(status);  
        // Behind the scenes, Hibernate executes (conceptually):  
        // SELECT * FROM events WHERE status = ? ORDER BY created_at DESC  
        // [status parameter bound safely]  
    }  
  
    /**  
     * Count events by status using prepared statement.  
     */  
    public long getEventCountByStatus(EventStatus status) {  
        return eventRepository.countByStatus(status);  
        // Becomes: SELECT COUNT(*) FROM events WHERE status = ?  
    }  
}
```

C. Concurrent processing and threads

This project demonstrates thread pooling, producer-consumer patterns, and thread-safe blocking queues to handle asynchronous event processing. Multiple worker threads safely process events concurrently without data corruption, race conditions, or thread collisions.

Key Concurrency Concepts Highlighted

Thread Pool Creation: Instead of spawning a new thread for each event (expensive), the application creates a fixed pool of 3 reusable threads once at startup. This dramatically improves performance under load.

Blocking Queue: A thread-safe queue allows the main application thread (producer) to safely drop events in, while worker threads (consumers) safely retrieve them—without explicit locks or synchronization code.

Volatile Flags: The running boolean is marked volatile to ensure all threads see the most recent value when the application shuts down, preventing stale reads across CPU cores.

Code Example: Thread Pool and Queue Setup

Bean Configuration

```
Java
@Configuration
public class ProcessorConfig {

    @Value("${worker.thread.count:3}")
    private int workerThreadCount;

    /**
     * Create a thread-safe blocking queue for events.
     * Multiple threads will safely put/take from this queue.
     */
}
```



```

    */
@Bean
public BlockingQueue<Event> eventQueue() {
    return new LinkedBlockingQueue<>();
    // LinkedBlockingQueue handles all synchronization internally
    // No explicit locks needed
}

/**
 * Create a fixed thread pool with exactly 3 reusable threads.
 * Much more efficient than creating a new thread per event.
 */
@Bean
public ExecutorService executorService() {
    return Executors.newFixedThreadPool(workerThreadCount);
    // Thread pool is created once at startup
    // Threads are reused for the lifetime of the application
}
}

```

Orchestration and Worker Queues

```

Java
@Service
public class EventProcessor {

    private final BlockingQueue<Event> eventQueue;
    private final ExecutorService executorService;
    private volatile boolean running = false; // Safe visibility across threads

    /**
     * Start exactly 3 worker threads that continuously process events.
     */
    public void startWorkers() {
        if (running) {
            log.warn("Workers are already running");
            return;
        }

        running = true;
    }
}

```

```

        log.info("Starting {} worker threads", workerThreadCount);

        // Submit 3 WorkerThread runnables to the thread pool
        for (int i = 0; i < workerThreadCount; i++) {
            executorService.execute(new WorkerThread(eventQueue, eventRepository,
i + 1));

            // Each worker thread now runs independently in the pool
            // All 3 threads share the same eventQueue
        }
    }

    /**
     * Producer: Submit an event to the queue for processing.
     * Thread-safe: main request thread safely adds events.
     */
    public void submitEvent(Event event) {
        try {
            // Save to database first (before queue) to avoid race conditions
            eventRepository.save(event);

            // Put event in queue - blocks if queue is full
            // Worker threads will see this event via take()
            eventQueue.put(event);

            log.info("Event {} submitted to queue", event.getId());
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log.error("Interrupted while submitting event", e);
        }
    }

    /**
     * Gracefully shutdown worker threads.
     */
    public void stopWorkers() {
        running = false;
        executorService.shutdown(); // Wait for all tasks to complete
    }

    /**
     * Get queue size - worker thread count can be inferred from queue depth.

```

```

    */
    public int getQueueSize() {
        return eventQueue.size(); // Thread-safe operation
    }
}

```

Consumer Processing Events:

```

Java
@Slf4j
public class WorkerThread implements Runnable {

    private final BlockingQueue<Event> eventQueue;
    private final EventRepository eventRepository;

    @Override
    public void run() {
        log.info("Worker {} started", workerId);

        try {
            while (!Thread.currentThread().isInterrupted()) {
                // Take() blocks until an event is available
                // Multiple workers safely pull from the same queue
                Event event = eventQueue.take();
                processEvent(event);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            log.info("Worker {} interrupted", workerId);
        }
    }

    private void processEvent(Event event) {
        // Each worker independently processes its assigned event
        // No locks needed - each event is owned by one worker
        event.setStatus(EventStatus.RUNNING);
        event.setStartedAt(LocalDateTime.now());
        eventRepository.save(event);

        // Simulate work
    }
}

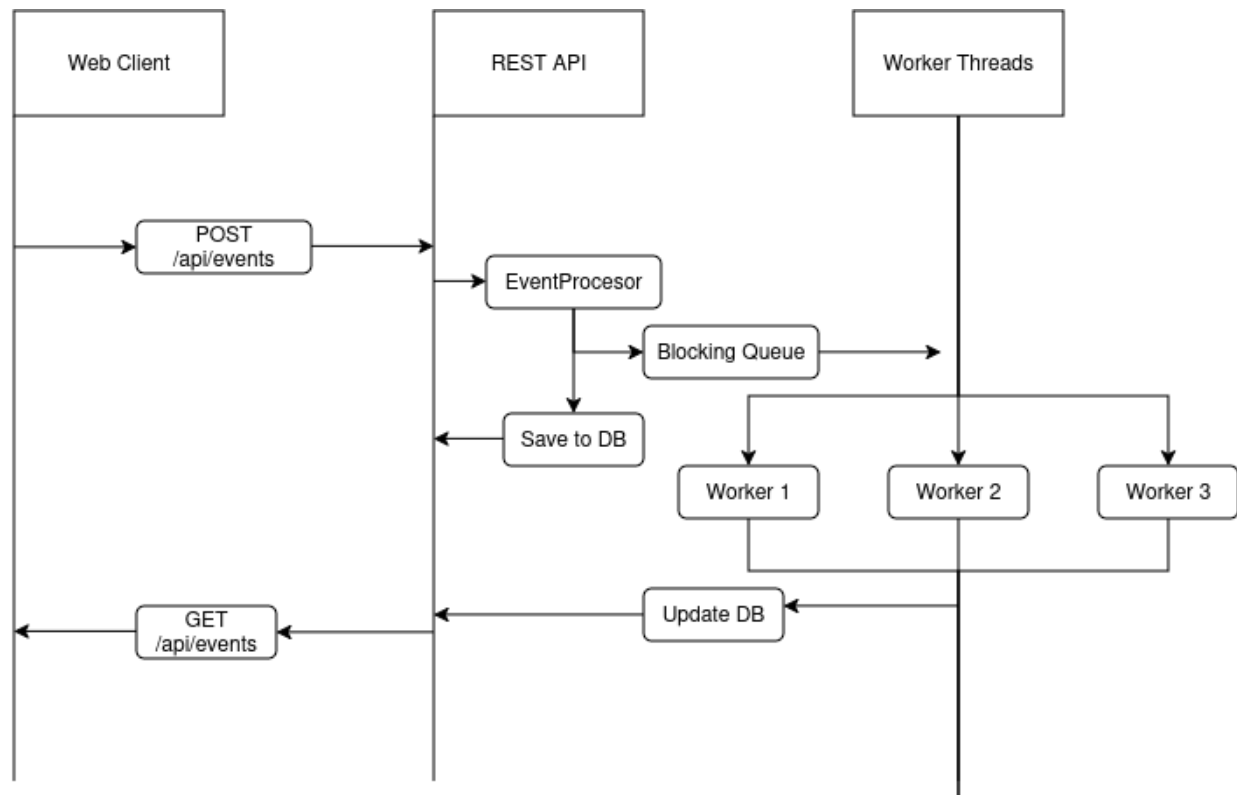
```

```
Thread.sleep(event.getProcessingTimeMs());

// Mark complete
event.setStatus(EventStatus.COMPLETED);
event.setCompletedAt(LocalDateTime.now());
eventRepository.save(event);
}
}
```

6. Architecture

This diagram illustrates the asynchronous event-processing architecture. The workflow begins when the **Web Client** initiates a **POST /api/events** request to the **REST API**. The **EventProcessor** handles the request by first persisting the initial event state to the database and then placing the event into a thread-safe **BlockingQueue**. Three independent **Worker Threads** monitor this queue, retrieving and processing events as they become available. Upon completion of each task, the workers update the database, allowing the **Web Client** to retrieve the finalized status via a subsequent **GET /api/events** request.



7. Conclusion

The Event-Driven Processing Simulator successfully demonstrates the core principles of asynchronous, multi-threaded system design. By leveraging the Producer-Consumer pattern and robust Java concurrency utilities like `ExecutorService` and `BlockingQueue`, this project achieves efficient task management and decoupling of system components. The integration of Spring Boot and JPA further highlights how modern frameworks can simplify complex backend operations, ensuring scalability, security, and maintainability. This project serves as a functional model for understanding how high-throughput systems can remain responsive and reliable under load, providing a valuable sandbox for testing architectural decisions in event-driven environments.